day03am

STL and Struct

Kittipitch Kuptavanich

2026-03-29

Grader

- C2E12: Hell's Kitchen Xtreme (66-Exam1)
- C2P07: Sort by displacement
- C2P08: A hand of Slave
- C2P09: Fan Meeting
- C2P19: Event Search

Vector of Struct

```
05 struct Student {
06   int id;
07   string name;
08   double gpa;
09 };
10
11 int main() {
12   vector<Student> students = {
13     {101, "Alice", 3.5},
14     {102, "Bob", 3.8},
15     {103, "Charlie", 2.9}
16   };
17
18   for (const auto& s : students) {
19     cout << s.id << " " << s.name << " " << s.gpa << "\n";
20   }
21   return 0;
22 }
```

```
.. // range-based loop, since C++11
18 for (declaration : expression) {
19   // loop body
20 }
```

```
Alice 3.5
Bob 3.8
Charlie 2.9
```

Iterator

- **Iterator เป็น pointer ที่ไว้ใช้เพื่อ Loop เข้าถึง STL containers แต่ละตัว**

```
05 int main() {
06   vector<int> v = {10, 20, 30, 40, 50};
07   vector<int>::iterator it;
08
09   for (it = v.begin(); it != v.end(); ++it) {
10     cout << *it << " ";
11   }
12   cout << endl;
13   return 0;
14 }
```

```
10 20 30 40 50
```

- `v.begin()` → **ชี้ไปที่ Element แรกของ Vector**
- `v.end()` → **ชี้ไปที่ Element หลังตัวสุดท้ายของ Vector (`v.end()` อยู่นอก Vector)**
- `*it` → **dereferences เพื่ออ่านค่าจาก iterator**

Reverse Traversal

```
int main() {
  vector<int> v = {10, 20, 30, 40, 50};
  vector<int>::iterator it;

  for (it = v.rbegin(); it != v.rend(); it++) {
    cout << *it << " ";
  }

  cout << endl;
  return 0;
}
```

```
50 40 30 20 10
```

- ไม่ควรเริ่มจาก `v.end()-1` แล้วทำ `it--` จน `>= v.begin()`
  - `>=` ใช้ได้กับ Vector แต่ไม่ support ใน container อื่น ๆ

Iterator and Struct

```
int main() {
  vector<Student> students = {
    {101, "Alice", 3.5},
    {102, "Bob", 3.8},
    {103, "Charlie", 2.9}
  };


  vector<Student>::iterator it;
  for (it = students.begin(); it != students.end(); it++) {
    cout << it->name << " " << it->gpa << "\n";
  }

  return 0;
}
```

```
Alice 3.5
Bob 3.8
Charlie 2.9
```

- **Dereferences เพื่ออ่านค่าจาก iterator ด้วย ->**

Map Traversal

```
10 int main() {
11   map<string, int> myMap = {
12     {"orange", 2},
13     {"apple", 3},
14     {"banana", 5}
15   };
16
17   // std::map automatically sorts keys in ascending order.
18   for (auto it = myMap.begin(); it != myMap.end(); it++) {
19     cout << it->first << " => " << it->second << endl;
20   }
21
22   return 0;
23 }
```

```
apple => 3
banana => 5
orange => 2
```

Iterator w/ a Lambda Function

```
03 #include <algorithm> // Required for find_if
.. ...
10 int main() {
11   vector<Student> students = {
12     {101, "Alice", 3.5},
13     {102, "Bob", 3.8},
14     {103, "Charlie", 2.9}
15   };
16
17   auto it = find_if(students.begin(), students.end(), [](const Student& s) {
18     return s.gpa < 3.0;
19   });
20
21   if (it != students.end()) {
22     cout << it->name << " " << it->gpa << "\n";
23   }
24   return 0;
25 }
```

```
Charlie 2.9
```

Iterator w/ a Lambda Function [2]

```
..  ...
17    auto it = students.begin(); // Find all
18    while (it != students.end()) {
19      it = find_if(it, students.end(), [](const Student& s) {
20        return s.gpa < 3.0;
21      });
22      if (it != students.end()) {
23        cout << it->name << " " << it->gpa << "\n";
24        ++it;  // Move to the next element to avoid infinite loop
25      }
26    }
..  ...
```

Deleting an Element Using Iterator

```
10 int main() {
11   vector<Student> students = {
12     {101, "Alice", 3.5},
13     {102, "Bob", 3.8},
14     {103, "Charlie", 2.9}
15   };
16
17   for (auto it = students.begin(); it != students.end(); ) {
18     if (it->id == 102) {
19       it = students.erase(it); // returns the next valid iterator
20     } else {
21       ++it;
22     }
23   }
24   return 0;
25 }
26
```

SORTING

The sort() Function

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
05 template <typename T>
06 void printVector(const vector<T>& vec)
07 {
08     for (const T& elem : vec) {
09         cout << elem << " ";
10     }
11     cout << endl;
12 }
13
14 int main()
15 {
16     vector<int> numbers = { -5, 2, -8, 1, 9, -3};
17
18     // Sort in ascending order
19     sort(numbers.begin(), numbers.end());
20     cout << "Ascending order: ";
21     printVector(numbers);
..  ...
```

```
Ascending order: -8 -5 -3 1 2 9
```

Comparison Function and Comparator

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
```

- ถ้าต้องเรียงจากมากไปน้อย (Descending)

```
    // Sort in descending order
    sort(numbers.begin(), numbers.end(), greater<int>());
    cout << "Descending order: ";
    printVector(numbers);
```

Comparator

- Other ways to compare
  - `greater<int>`
  - `greater_equal<T>`
  - ...

```
Descending order: 9 2 1 -3 -5 -8
```

Comparison Function and Comparator [2]

- ถ้าต้องการเรียงด้วยค่า absolute?

```
// Sort by absolute values
    sort(numbers.begin(), numbers.end(), [](int a, int b) {
        return abs(a) < abs(b);
    });
    cout << "Sorted by absolute values: ";
    printVector(numbers);
```

Comparison Function (Lambda)

- Comparison Function (ไม่เหมือนกรณี `qsort()` ใน C)
  - คืนค่า `true` ก็ต่อเมื่อ *a* <u>มาก่อน</u> *b* ในการเรียงลำดับที่ต้องการ
  - คืนค่า `false` ในกรณีอื่น ๆ

Comparison Function

- **Function Signature:**

```
// typical signature
bool comparison_f(const T& a, const T& b);

// lambda function
[](const T& a, const T& b) -> bool {
    // Comparison logic here
}
```

Comparison Function vs Comparator

```
#include <cstdlib> // For abs

// Comparison function
bool comp(int a, int b) {
  return abs(a) < abs(b);
}



int main() {
  vector<int> vec = {-5, 2, -3, 1, -4};
  sort(vec.begin(), vec.end(), comp);
  cout << "Sorted by absolute values: ";
  for (int x : vec) {
    cout << x << " ";
  }
  cout << "\n";
  return 0;
}
```

```
#include <cstdlib> // For abs

// Comparator struct
struct Comp {
  bool operator()(int a, int b) const {
    return abs(a) < abs(b);
  }
};

int main() {
  vector<int> vec = {-5, 2, -3, 1, -4};
  sort(vec.begin(), vec.end(), Comp());
  cout << "Sorted by absolute values: ";
  for (int x : vec) {
    cout << x << " ";
  }
  cout << "\n";
  return 0;
}
```

Sorting w/ Data Structures

```
05 template <typename T>
06 void printVector(const vector<T>& vec)
07 {
08     for (const T& elem : vec) {
09         cout << elem.first << "," << elem.second << " ";
10     }
11     cout << endl;
12 }
13
14 int main()
15 {
16     vector<pair<char, int>> pairs = {
17         {'a', 2}, {'b', 3}, {'c', 2}, {'d', 1}, {'e', 1}, {'f', 3},
18     };
19     // sort by the second element
20     sort(pairs.begin(), pairs.end(), [](const auto & lhs, const auto & rhs) {
21         return lhs.second < rhs.second;
22     });
.. ...
```

```
d,1 e,1 a,2 c,2 b,3 f,3
```

Sorting w/ Data Structures [2]

```
05      vector<pair<char, int>> pairs = {
06          {'a', 2}, {'b', 3}, {'c', 2}, {'d', 1}, {'e', 1}, {'f', 3},
07          {'g', 1}, {'h', 3}, {'i', 3}, {'j', 2}, {'k', 3}, {'l', 2},
08          {'m', 1}, {'n', 3}, {'o', 1}, {'p', 3}, {'q', 1}, {'r', 1},
09          {'s', 3}, {'t', 2}, {'u', 3}, {'v', 3}, {'w', 2}, {'x', 1},
10          {'y', 1}, {'z', 1}
11
12      };
13      // sort by the second element
14      sort(pairs.begin(), pairs.end(), [](const auto & lhs, const auto & rhs) {
15          return lhs.second < rhs.second;
16      });
17
18      printVector(pairs);
19
20
21      z,1 r,1 d,1 e,1 y,1 g,1 x,1 q,1 o,1 m,1
22      t,2 a,2 l,2 j,2 w,2 c,2 v,3 u,3 n,3 s,3
..      p,3 k,3 i,3 h,3 f,3 b,3
```

สังเกตว่า ตำแหน่งเปลี่ยนไปจาก input z มาอยู่หน้าสุด (Instability)

The `sort()` Function [2]

- **NOT a STABLE sorting algorithm (ไม่รับประกันว่าลำดับจะคงเดิม กรณี key เท่า)**
  - **ถ้าต้องการ stability ให้ไปใช้ `stable_sort()` (w/ speed Trade-off)**
- **In-place sorting algorithm -> memory efficient**
- **Time Complexity**
  - **Best-case time complexity**: $O(n \log n)$
  - **Average-case time complexity**: $O(n \log n)$
  - **Worst-case time complexity**: $O(n \log n)$
- **ใช้ Introsort algorithm ที่เป็นลูกผสมระหว่าง of Quicksort and Heapsort (ไม่ cover ในค่ายนี้)**

Comparison Function [2]

สามารถ Implement operator < (เครื่องหมายน้อยกว่า ไว้ใน struct และจะโดนเรียกใช้โดยอัตโนมัติเมื่อมีการเปรียบเทียบค่า ของ Point

```
05 struct Point {
06     double x;
07     double y;
08
09     // descending order of displacement from the origins
10     bool operator<(const Point& other) const {
11         return other.x * other.x + other.y * other.y < x * x + y * y;
12     }
13 };
14
15 int main() {
16     // Create a vector of Point structs
17     vector<Point> points = {{3.0, -4.0}, {1.0, 1.0}, { -5.0, 2.0}, {0.0, 0.0}};
18
19     // Using the comparator inside the struct
20     sort(points.begin(), points.end());
21
22
..
```

```
(-5, 2)
(3, -4)
(1, 1)
(0, 0)
```

ไม่ส่ง Comparator Function ทำให้มีการเรียกใช้ Default Operator <

Comparison Function [3]

- **Priority Queue w/ built-in struct comparator**

```
    vector<Point> points = {{3.0, -4.0}, {1.0, 1.0}, { -5.0, 2.0}, {0.0, 0.0}};

    // Define a priority queue of Points using the < operator defined in Point struct
    priority_queue<Point> pq(points.begin(), points.end());

    // Output of Max priority queue is reverse of sorted()
    while (!pq.empty()) {
        Point point = pq.top();
        cout << "(" << point.x << ", " << point.y << ")" << endl;
        pq.pop();
    }
...
```

ไม่ส่ง **Comparison Function** ทำให้มีการเรียกใช้ **Default Operator** **<**

```
(0, 0)
(1, 1)
(3, -4)
(-5, 2)
```

Comparison Function [4]

- **Priority Queue w/ a comparator outside the struct**

```
// Define a priority queue of Points using your own pointComparator function
priority_queue<Point, vector<Point>, pointComparator> pq(points.begin(), points.end());

```

ต้องระบุ data struct ที่จะใช้สร้าง Priority Queue <u>ถ้าจะใช้ Custom Comparator</u> โดยปกติ default จะเป็น vector

Custom Comparator กรณีไม่ต้องการใช้ Operator < ที่มากะ Struct หรือไม่มีมาให้

```
// Custom comparator function for Point struct
bool pointComparator(const Point& p1, const Point& p2)
{
    return p1.x * p1.x + p1.y * p1.y > p2.x * p2.x + p2.y * p2.y;

}
```

SEARCHING

Searching

- หนึ่งในปัญหาที่พบบ่อยที่สุดใน Computing
- ใน Class นี้เราจะพิจารณาในแง่ที่ว่า Item ที่ต้องการ เป็นสมาชิกใน Collection นั้น ๆ หรือไม่
  - True หรือ False
  - สามารถปรับ Algorithm เพื่อใช้บอกตำแหน่งที่พบได้

Searching [2]

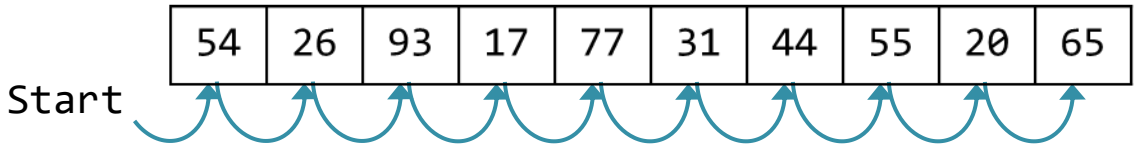
- ใน C++ STL การตรวจสอบว่า Item ใด ๆ อยู่ใน List หรือไม่ ทำได้โดยการใช้ `find()` Function

```
vector<int> v = {1, 2, 3, 4, 5};
int key = 3;
vector<int>::iterator it = find(v.begin(), v.end(), key); // linear search

if (it != v.end()) {
  int index = it - v.begin();
  cout << "Element " << key << " found at index " << index << endl;
} else {
  cout << "Element " << key << " not found" << endl;
}
```

- พิจารณาถึงกระบวนการที่เกิดขึ้นเบื้องหลังเพื่อที่จะได้ผลลัพธ์ดังกล่าว พบมีวิธีในการ Search ที่แตกต่างกัน

Linear Search

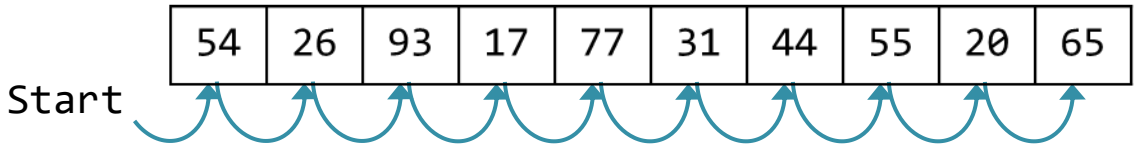


Or Sequential Search

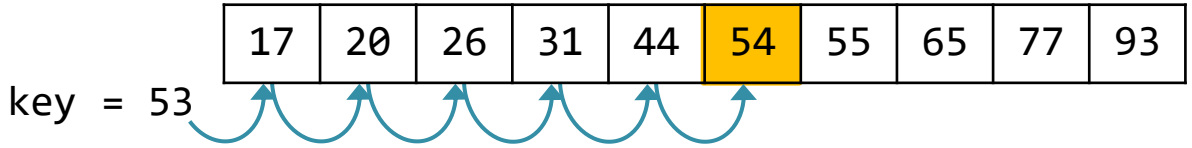
- ไล่ดูทีละ **Element** จนพบค่าที่ต้องการ

```
function searchArray(array, key)
  for i ← 0 to length(array) - 1 do
    if array[i] equals key then
      return true
    end if
  end for
  return false
end function
```

Linear Search [2]



- ในกรณีที่ List ไม่ได้เรียงตามลำดับ (Unordered List) จะต้องเปรียบเทียบค่ากับทุก Element จนกว่าจะพบ
- กรณี List เรียงตามลำดับ (Ordered List)



  - พิจารณาเฉพาะ Element ที่น้อยกว่าหรือเท่ากับ key
    - เมื่อพบ Element ที่มากกว่า key → หยุดค้นหา

Binary Search

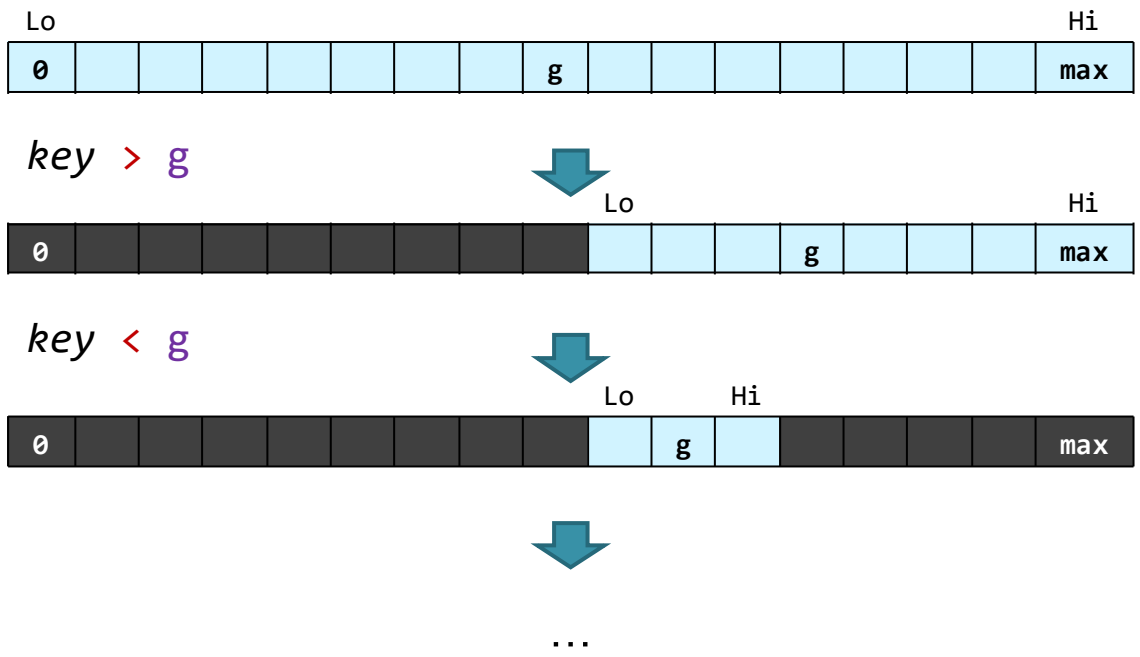
- พิจารณาการเปิดหาคำศัพท์ *key* ใน **dictionary**
  - ค่าแต่ละค่าในช่วงที่ต้องการหาเป็นค่าที่<u>เรียงกัน</u>
  - หากสุ่มเปิดมาหน้าที่อยู่ในช่วงตัวอักษร<u>*ก่อน*</u>คำที่ต้องการหา
    - แสดงว่าหน้านั้นและ <u>ทุกหน้าก่อนนั้น</u>ไม่มีคำตอบ
    - หาคำตอบในหน้าหลังจากนั้นเท่านั้น

- สุ่มตำแหน่ง (Page) g ที่กึ่งกลาง ถ้า *key* > g แสดงว่า g มีค่าน้อยไป
- คำตอบอยู่ในช่วงทางด้านขวาของ g (ตัดช่วงที่ไม่ใช่คำตอบทิ้งทีละครึ่ง)

Binary Search [2]

หรือ Bisection Search

- **เราสามารถใช้ Algorithm การ Search นี้กับการ Search หา Element ใน <u>Ordered List</u>**



Binary Search [3]

- สมมติเรามี ***sorted*** `list` (array)

10	11	12	16	18	23	29	33	48	54	57	68	77	84	98

- ต้องการหา key = **23**

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]
lo							mid							hi
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98
lo			mid			hi								
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98
				lo	mid	hi								
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98

Binary Search [4]

10	11	12	16	18	23	29	33	48	54	57	68	77	84	98

- ต้องการหา key = 50

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]
lo							mid							hi
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98
								lo			mid			hi
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98
								lo	mid	hi				
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98
								lo mid hi						
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98

กรณีหาไม่เจอ จะจบที่ index ของ Element ที่มีค่ามากที่สุดที่ไม่เกิน key

Binary Search [5]

```
function BinarySearch(A, value)
    low ← 0
    high ← length(A) - 1

    while low ≤ high do
        mid ← (low + high) / 2
        if A[mid] = value then
            return true
        else if A[mid] < value then
            low ← mid + 1
        else
            high ← mid - 1
        end if
    end while

    return false
end function
```

Binary Search [6]

- ใช้ได้เฉพาะกรณี input เรียงลำดับมาแล้วเท่านั้น
- Time Complexity (Big O):
  - Worst-case: $O(\log n)$
  - Average-case: $O(\log n)$
  - Best-case: $O(1)$
- Space Complexity:
  - $O(1)$

The binary_search() Function

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
int main()
{
    // Define a sorted vector
    vector<int> v = { 10, 11, 22, 16, 18, 23, 29, 33, 48, 54, 57, 68, 77, 84, 98 };

    // Perform binary search
    int key = 23;
    bool found = binary_search(myVector.begin(), myVector.end(), key);

    // Check if the key element is found
    if (found) {
        cout << key << " found" << endl;
    } else {
        cout << key << " not found" << endl;
    }

    return 0;
}
```

```
23 found
```

The lower_bound() Function

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]
10	11	12	16	18	23	29	33	48	54	57	68	77	84	98

```
int main()
{
    // Define a sorted vector
    vector<int> v = { 10, 11, 22, 16, 18, 23, 29, 33, 48, 54, 57, 68, 77, 84, 98 };

    // Perform binary search
    int key = 23;
    auto it = lower_bound(v.begin(), v.end(), key);

    // Check if the key element is found
    if (it != v.end() && *it == key) {
        int index = it - v.begin();
        cout << key << " found at index " << index << endl;
    } else {
        int index = it - v.begin();
        cout << key << " not found" << endl;
    }

    return 0;
}
```

```
23 found at index 5
```

The `lower_bound()` Function [2]

- ใช้ **Binary Search**
- คืนค่า **Pointer** ที่ชี้ไปยังสมาชิกแรกที่มีค่าไม่น้อยกว่า ***key*** (ในช่วงที่ระบุ) แยกได้เป็น **2** กรณี
  - ถ้า ***key*** เจอในช่วงดังกล่าว จะคืน **Pointer** ที่ชี้ไปยัง ***key***
    - ถ้ามี **key** ซ้ำในหลายตำแหน่ง จะเป็น **Address** ของ **Element แรก**ที่เจอ (**index** ต่ำสุด)
    - เช่นหา **5** ใน `{5, 5, 5, 5}` จะได้ **Pointer** ไปตำแหน่ง **index 0**
  - ถ้าหา ***key*** ไม่เจอในช่วงดังกล่าว จะคืน **Pointer** ที่ชี้ไปยัง **Element** ที่มีค่าน้อยที่สุดที่มากกว่า ***key***
    - เราสามารถนำ ***key*** มา **insert** เข้า ณ ตำแหน่งนี้ เพื่อให้ผลลัพธ์ยังเรียงลำดับที่ถูกต้อง
    - เช่น หา **5** ใน `{2, 4, 6, 8}` จะคืนค่า **Pointer** ชี้ไปที่เลข **6** (**index** ที่ **2**)

The lower_bound() Function [3]

```
10      int key1 = 5, key2 = 7;
11      vector<int> v1 = {5, 5, 5, 5, 5};
12      vector<int> v2 = {0, 2, 4, 6, 8};
13
14      auto it1 = lower_bound(v1.begin(), v1.end(), key1);
15      int index = it1 - v1.begin();
16      if (*it1 == key1) {
17          cout << key1 << " found at index " << index << endl;
18      } else {
19          cout << key1 << " not found but can be inserted at index " << index << endl;
20      }
21
22      auto it2 = lower_bound(v2.begin(), v2.end(), key2);
23      index = it2 - v2.begin();
24      if (*it2 == key2) {
25          cout << key2 << " found at index " << index << endl;
26      } else {
27          cout << key2 << " not found but can be inserted at index " << index << endl;
28      }
..  ...
```

[0]	[1]	[2]	[3]	[4]
5	5	5	5	5

[0]	[1]	[2]	[3]	[4]
0	2	4	6	8

```
5 found at index 0
7 not found but can be inserted at index 4
```

The lower_bound() Function [4]

```
23    Point key = {5, 1};
24
25    // Define a sorted vector of points
26    vector<Point> v1 = {{5, 2}, {5, 4},
27                        {5, 0}, {5, 3}, {5, 1}};
28
29    // Sort v1 w/ the struct's comparator
30    sort(v1.begin(), v1.end());
31
32    // Perform binary search on v1
33    auto it1 = lower_bound(v1.begin(), v1.end(), key);
34    int index1 = distance(v1.begin(), it1);
35
36    if (it1 != v1.end() && (it1->x == key.x  && it1->y == key.y)) {
37      cout << "Found: Point {" << key.x << ", " << key.y
38           << "} is at index " << index1 << endl;
39    } else {
40      cout << "NOT Found: Point {" << key.x << ", " << key.y
41           << "} can be inserted at index " << index1  << endl;
42    }
..  ...
```

```
05 struct Point {
06   int x;
07   int y;
08
09   bool operator<(const Point& other) const
10   {
11     // compare using displacement from (0,0)
12     return x * x + y * y <
13            other.x * other.x + other.y * other.y;
14   }
15 };
```

สามารถนิยาม
operator==(const Point& other)
มาใช้เปรียบเทียบแทนได้

The lower_bound() Function [5]

```
05 // Define a custom comparator function
06 // outside the struct
07
08 bool cmp(const Point& p1, const Point& p2)
09 {
10   // Compare based on displacement (0,0)
11   return p1.x * p1.x + p1.y * p1.y <
12          p2.x * p2.x + p2.y * p2.y;
13 }
14
```

```
23   Point key = {5, 1};
24   vector<Point> v1 = {{5, 2}, {5, 4},
25                       {5, 0}, {5, 3}, {5, 1}};
26
27   // Sort the vector using the custom comparator
28   sort(v1.begin(), v1.end(), cmp);
29
30   // Perform binary search on the vector
31   auto it1 = lower_bound(v1.begin(), v1.end(), key, cmp);
32   int index1 = distance(v1.begin(), it1); // Calculate the index
33
34   // Output the result of the search
35   if (it1 != v1.end() && (it1->x == key.x && it1->y == key.y)) {
36     cout << "Found: Point {" << key.x << ", " << key.y
37        << "} is at index " << index1 << endl;
38   } else {
39     cout << "NOT Found: Point {" << key.x << ", " << key.y
40        << "} can be inserted at index " << index1  << endl;
41   }
.. ...
```